



Project Real-Time Embedded Systems

Élève :
Bolteau Maeva

Niveau : Aéro4

Mardi 28 avril 2026
Année universitaire 2025-2026

Introduction

This final project is part of the study of **Real-Time Embedded Systems** class. The primary objective is to design and implement simulation algorithms capable of managing a set of concurrent tasks, ensuring that timing constraints are strictly respected while optimizing processor usage.

To achieve this, we have structured our study into three major stages :

- The **characterization of task** τ_1 , dedicated to the statistical analysis (Min, Max, Quartiles) of a computational task involving large random numbers to determine its Worst-Case Execution Time (WCET).
- The implementation of a **non-preemptive scheduling** algorithm at the job level, aimed at finding a valid schedule that prevents deadline misses while minimizing total waiting time and maximizing processor idle time.
- The exploration of an **optimized scenario** where the scheduling logic is adapted to further minimize waiting time, specifically by evaluating the impact of allowing task τ_5 to miss its deadline.

The entire development was carried out in the **Python** and **C** language.

Table des matières

1	Schedulability Analysis and Theoretical Framework	3
1.1	Task Model and Constraints	3
1.2	Feasibility Conditions	3
1.3	Non-Preemptive Response Time Analysis	3
1.4	Hyperperiod Determination	4
1.5	Optimization Strategy	4
1.6	Experimental Results for Task τ_1	4
1.7	Final WCET and Schedulability Verification	5
2	Implementation and Algorithms	6
2.1	Scenario A : Strict Earliest Deadline First (EDF)	6
2.2	Scenario B : Optimized Waiting Time (T5 Miss Allowed)	7
3	Conclusion	8

1 Schedulability Analysis and Theoretical Framework

This section establishes the theoretical foundation for the scheduling of our task set, utilizing the formulas and conditions defined in the real-time systems curriculum.

1.1 Task Model and Constraints

The system follows a periodic task model where each task τ_i is characterized by (C_i, T_i) with :

- $\tau_1 : (C_1, 10)$
- $\tau_2 : (3, 10)$
- $\tau_3, \tau_4 : (2, 20)$
- $\tau_5, \tau_6 : (2, 40)$
- $\tau_7 : (3, 80)$

1.2 Feasibility Conditions

According to the course, the processor utilization factor U must satisfy the necessary condition for schedulability :

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq 1$$

Applying the values from the assignment :

$$U = \frac{C_1}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80}$$

$$U = \frac{C_1}{10} + \frac{3}{10} + \frac{4}{20} + \frac{4}{40} + \frac{3}{80} = \frac{C_1}{10} + \frac{51}{80}$$

For the system to be potentially schedulable, C_1 must not exceed 3.625 units of time.

1.3 Non-Preemptive Response Time Analysis

Since the assignment requires a **non-preemptive schedule**, we must account for the **Blocking Time** (B_i). A high-priority task can be blocked by a lower-priority task that has already started its execution.

The Worst-Case Response Time (WCRT) R_i for a task is calculated by using this formula :

$$R_{ij}^n = C_{ij}^n + \max(R_{ij}^{n-1} - (a_{ij}^n - a_{ij}^{n-1}), 0)$$

In our specific non-preemptive case, the blocking term for a task τ_i is defined by the longest execution time of all tasks with lower priority :

$$B_i = \max_{j>i} (C_j)$$

For τ_1 , the blocking time is $B_1 = \max(3, 2, 2, 2, 2, 3) = 3$. Thus, even the highest priority task must be able to tolerate a delay of 3 units without missing its deadline of 10.

1.4 Hyperperiod Determination

The study of the schedule is conducted over the Hyperperiod, defined as $H = \text{lcm}(T_1, \dots, T_7)$.

$$H = \text{lcm}(10, 20, 40, 80) = 80$$

This interval contains :

- 8 jobs of τ_1 and τ_2
- 4 jobs of τ_3 and τ_4
- 2 jobs of τ_5 and τ_6
- 1 job of τ_7

Total jobs to schedule : $8 + 8 + 4 + 4 + 2 + 2 + 1 = 29$ jobs.

1.5 Optimization Strategy

The search algorithm aims to minimize the **Total Waiting Time** :

$$\text{Minimize } \sum (S_{ij} - a_{ij})$$

where S_{ij} is the start time and a_{ij} is the arrival time. By minimizing this delay, we effectively maximize the contiguous idle time of the processor, as requested.

1.6 Experimental Results for Task τ_1

The execution of the `multiplication.c` script allowed for a precise characterization of τ_1 's temporal behavior. The task performed 10^6 iterations of large number multiplications to simulate a realistic computational load. The statistical distribution of the execution times (in microseconds) is summarized below :

```
maevabolteau@OrdiMava:/mnt/c/aero4/systeme_temps_reel/final_repo$ ./multiplication2
Min: 0.016000 ms
Max: 2.007000 ms
Q1 : 0.016000 ms
Q2 : 0.016000 ms
Q3 : 0.017000 ms
```

FIGURE 1 – Results of the experimentation

1.7 Final WCET and Schedulability Verification

Based on the experimental results, the measured Worst-Case Execution Time for τ_1 is 2.007. To ensure system robustness and account for hardware variability, a safety margin of 20% is applied :

$$C_1 = 2.007 \times 1.20 = 2.4084$$

This value $C_1 = 2.4084$ is used for the final schedulability analysis. The total processor utilization factor U is calculated as follows :

$$U = \frac{2.4084}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80}$$

$$U = 0.87834$$

Since $U \approx 0.878 < 1$, the system satisfies the necessary condition for schedulable task sets.

2 Implementation and Algorithms

Two Python scripts were developed to simulate the execution order. Both scripts generate 29 total jobs across the 80 ms hyper-period.

2.1 Scenario A : Strict Earliest Deadline First (EDF)

In this scenario, the priority is strictly given to the job with the closest absolute deadline. If deadlines are equal, the Shortest Job First (SJF) is applied to reduce waiting time.

- **Result** : All 29 jobs met their deadlines.
- **Total Waiting Time** : 157.1528
- **Total Processor Idle Time** : 9.7328 ms

nal_repo/scheduler_no_miss.py					
Start	Job	End	DL	R.Time	Statu

t=0	J1,1	2.4084	10	2.4084	OK
t=2.4084	J2,1	5.4084	10	5.4084	OK
t=5.4084	J3,1	7.4084	20	7.4084	OK
t=7.4084	J4,1	9.4084	20	9.4084	OK
t=9.4084	J5,1	11.4084	40	11.4084	OK
t=11.4084	J1,2	13.8168	20	3.8168	OK
t=13.8168	J2,2	16.8168	20	6.8168	OK
t=16.8168	J6,1	18.8168	40	18.8168	OK
t=18.8168	J7,1	21.8168	80	21.8168	OK
t=21.8168	J1,3	24.2252	30	4.2252	OK
t=24.2252	J2,3	27.2252	30	7.2252	OK
t=27.2252	J3,2	29.2252	40	9.2252	OK
t=29.2252	J4,2	31.2252	40	11.2252	OK
t=31.2252	J1,4	33.6336	40	3.6336	OK
t=33.6336	J2,4	36.6336	40	6.6336	OK
t=40	J1,5	42.4084	50	2.4084	OK
t=42.4084	J2,5	45.4084	50	5.4084	OK
t=45.4084	J3,3	47.4084	60	7.4084	OK
t=47.4084	J4,3	49.4084	60	9.4084	OK
t=49.4084	J5,2	51.4084	80	11.4084	OK
t=51.4084	J1,6	53.8168	60	3.8168	OK
t=53.8168	J2,6	56.8168	60	6.8168	OK
t=56.8168	J6,2	58.8168	80	18.8168	OK
t=60	J1,7	62.4084	70	2.4084	OK
t=62.4084	J2,7	65.4084	70	5.4084	OK
t=65.4084	J3,4	67.4084	80	7.4084	OK
t=67.4084	J4,4	69.4084	80	9.4084	OK
t=70	J1,8	72.4084	80	2.4084	OK
t=72.4084	J2,8	75.4084	80	5.4084	OK

TOTAL WAITING TIME : 157.1528					
Total Processor Idle Time : 9.7328					

FIGURE 2 – Results of the no deadline miss scenario

2.2 Scenario B : Optimized Waiting Time (T5 Miss Allowed)

To minimize the global "Total Waiting Time", task τ_5 is deprioritized and so allowed to miss a deadline. For that, τ_5 is assigned to a virtual late priority, like this other tasks can execute sooner, and reducing their stay in the ready queue.

- **Result** : All 29 jobs met their deadlines even τ_5 .
- **Total Waiting Time** : 158.1528 ms
- **Total Processor Idle Time** : 9.7328 ms

nal_repo/scheduler_T5_miss.py					
Start	Job	End	DL	R.Time	Status

t=0	J1,1	2.4084	10	2.4084	OK
t=2.4084	J2,1	5.4084	10	5.4084	OK
t=5.4084	J3,1	7.4084	20	7.4084	OK
t=7.4084	J4,1	9.4084	20	9.4084	OK
t=9.4084	J6,1	11.4084	40	11.4084	OK
t=11.4084	J1,2	13.8168	20	3.8168	OK
t=13.8168	J2,2	16.8168	20	6.8168	OK
t=16.8168	J7,1	19.8168	80	19.8168	OK
t=19.8168	J5,1	21.8168	40	21.8168	OK
t=21.8168	J1,3	24.2252	30	4.2252	OK
t=24.2252	J2,3	27.2252	30	7.2252	OK
t=27.2252	J3,2	29.2252	40	9.2252	OK
t=29.2252	J4,2	31.2252	40	11.2252	OK
t=31.2252	J1,4	33.6336	40	3.6336	OK
t=33.6336	J2,4	36.6336	40	6.6336	OK
t=40	J1,5	42.4084	50	2.4084	OK
t=42.4084	J2,5	45.4084	50	5.4084	OK
t=45.4084	J3,3	47.4084	60	7.4084	OK
t=47.4084	J4,3	49.4084	60	9.4084	OK
t=49.4084	J6,2	51.4084	80	11.4084	OK
t=51.4084	J1,6	53.8168	60	3.8168	OK
t=53.8168	J2,6	56.8168	60	6.8168	OK
t=56.8168	J5,2	58.8168	80	18.8168	OK
t=60	J1,7	62.4084	70	2.4084	OK
t=62.4084	J2,7	65.4084	70	5.4084	OK
t=65.4084	J3,4	67.4084	80	7.4084	OK
t=67.4084	J4,4	69.4084	80	9.4084	OK
t=70	J1,8	72.4084	80	2.4084	OK
t=72.4084	J2,8	75.4084	80	5.4084	OK

TOTAL WAITING TIME : 158.1528					
Total Processor Idle Time : 9.7328					

FIGURE 3 – Results of the scenario where τ_5 is allowed to miss a deadline

This new scenario should reduce the the global "Total Waiting Time". But as, it shows it is not the case. A scheduler with the Shortest Job First can be very powerful in that case but the issue is some other Jobs will miss their deadline not only τ_5 .

3 Conclusion

This project successfully demonstrated the complete workflow for real-time system design, from task characterization to scheduling simulation.

The study reached the following conclusions :

- **System Feasibility** : Experimental analysis of τ_1 established a WCET of 2.007 ms[cite : 86]. With a 20% safety margin ($C_1 = 2.4084$), the processor utilization $U \approx 0.878$ remains within schedulable limits.
- **Scheduling Efficiency** : The Strict EDF algorithm (Scenario A) proved optimal, meeting all 29 deadlines with a total waiting time of 157.1528 ms.
- **Optimization Limits** : Attempting to deprioritize τ_5 (Scenario B) did not improve performance, resulting in a slightly higher waiting time of 158.1528 ms.

In conclusion, the **strict EDF approach** is the most effective strategy for this task set. Future developments should focus on testing these algorithms within a hardware-based RTOS environment to account for real-world execution overheads.